

Raw Data

#####

Timed Test (1 thread)

real 1m14.153s

user 1m14.082s

sys 0m0.049s

Timed Test (2 thread)

real 0m36.672s

user 1m13.243s

sys 0m0.071s

Timed Test (3 thread)

real 0m24.532s

user 1m13.522s

sys 0m0.051s

Timed Test (4 thread)

real 0m18.459s

user 1m13.673s

sys 0m0.124s

Timed Test (5 thread)

real 0m14.817s

user 1m13.990s

sys 0m0.064s

Timed Test (6 thread)

real 0m12.343s

user 1m14.001s

sys 0m0.029s

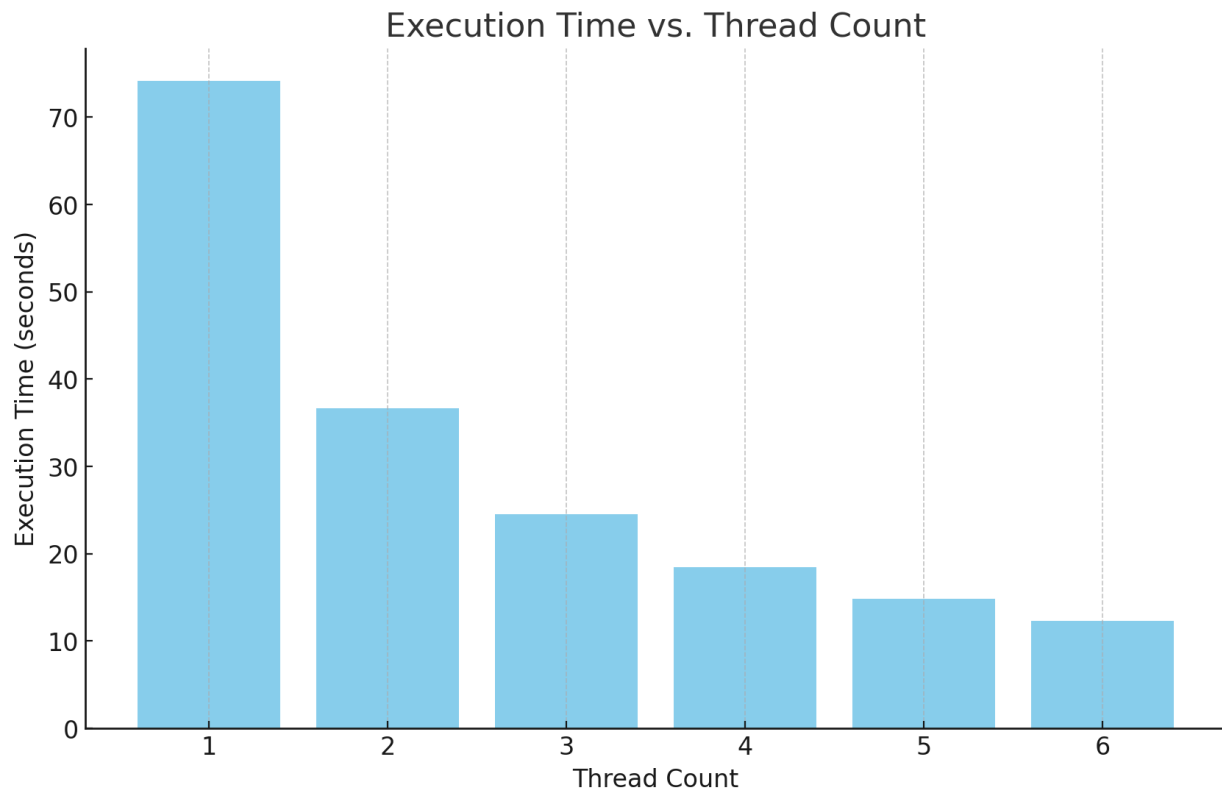
Computing Original Speedup

Thread Amount	Timed Test	Speed Up
1	1m14.153s	N/A
2	0m36.672s.	2.02x
3	0m24.532s	3.02x
4	0m18.459s	4.02x
5	0m14.817s	5.00x
6	0m12.343s	6.01x

Summary of Program

Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 46 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 8
On-line CPU(s) list: 0-7
Vendor ID: GenuineIntel
Model name: 13th Gen Intel(R) Core(TM) i7-13700K
CPU family: 6
Model: 183
Thread(s) per core: 1
Core(s) per socket: 8

Socket(s): 1



As seen above the execution time drops as the number of threads increases. This is because each thread gets to work on a portion of the range of numbers somewhat simultaneously, i.e. more threads more sharing of work.

Lock Removal / Duck Count with no Mutual Exclusion

```
student@VirtualBox:~/Downloads/Project_5A$ make
gcc -Wall -pedantic -pthread -g -o duckNums duckNums.c
student@VirtualBox:~/Downloads/Project_5A$ ./duckNums -th 6 -lm 2000000000
Number of Duck Counts: 767822094
student@VirtualBox:~/Downloads/Project_5A$ ./duckNums -th 6 -lm 2000000000
Number of Duck Counts: 767913705
student@VirtualBox:~/Downloads/Project_5A$ ./duckNums -th 6 -lm 2000000000
Number of Duck Counts: 767924640
student@VirtualBox:~/Downloads/Project_5A$ ./duckNums -th 6 -lm 2000000000
Number of Duck Counts: 767830599
student@VirtualBox:~/Downloads/Project_5A$ ./duckNums -th 6 -lm 2000000000
Number of Duck Counts: 767900097
student@VirtualBox:~/Downloads/Project_5A$ ./duckNums -th 6 -lm 2000000000
Number of Duck Counts: 767979315
```

When the locks were removed there was no mutual exclusion for the critical sections of the program. These sections were the parts where the program updated the shared resources, i.e our global variables, **currentCheck** and **duckNumberCount**. Not having mutual exclusion for these parts resulted in a race condition where multiple threads updated the global variable resulting in an uncoordinated access of these variables between threads.

Duck Count Update Relocation

#####

Timed Test (1 thread)

real 1m21.189s
user 1m21.139s
sys 0m0.030s

Timed Test (2 thread)

real 2m24.464s
user 4m9.305s
sys 0m37.069s

Timed Test (3 thread)

real 2m31.162s
user 5m19.251s
sys 1m46.553s

Timed Test (4 thread)

real 2m25.525s
user 6m18.332s
sys 2m30.428s

Timed Test (5 thread)

real 2m38.981s
user 7m10.527s
sys 4m4.513s

Timed Test (6 thread)

real 2m50.933s
user 8m33.686s
sys 5m49.211s

Making the threads increment the global duck counter within every block increases the overall time it takes for each thread to complete because each thread has to hold a lock on the counter whenever it finds a duck number. And since there are a lot of duck numbers the threads are probably waiting for the holding thread to release the lock when it tries to update the counter.